Read Protocol

> ***Learning goal.*** *Trace the smallest complete application protocol in the project and use it to practice actor messages, transaction states, local state access, and timeout handling.*

Implementation inspected: `origin/feat/readTransaction` at `1ed58b6`.
Integration status: implemented on a feature branch, absent from `feature/electionTransaction` at `d1222a2`.

1. Why read is the best first protocol

A read has one request hop and one reply hop. It does not involve the coordinator, a quorum, epochs, or recovery. That makes it a good place to learn the transaction framework before studying writes.

The protocol asks one replica: “What value is currently in your local array at this index?”

2. Message vocabulary

`ReadTransaction` defines three messages:

- `ReadMsg(transactionId, epochPair, sender, index)` travels from client to replica.
- `ReadResultMsg(transactionId, epochPair, sender, value, replicaId)` travels back.
- `ReadTimeoutMsg(transactionId, epochPair, sender)` is scheduled locally to the client.

The result contains the numeric replica ID because `ActorRef` alone is not the result format required by `AbstractClient.ReadResult`.

3. FSM states

The read FSM has:

```
INIT -> WAITING_RESULT -> DONE
                       -> TIMEOUT
```

- `INIT`: object constructed but request not sent.
- `WAITING_RESULT`: `ReadMsg` sent and timeout scheduled.
- `DONE`: valid result converted to a public callback.
- `TIMEOUT`: deadline expired first.

There is no retry or election transition in this transaction. A timeout ends this client request.

4. Happy-path trace

Assume client C reads index 3 from replica R2.

1. Application sends `ReadRequest(3, R2)` to C.
2. `AbstractClient.onReadRequest` calls `Client.sendRead(R2, 3)`.
3. C allocates transaction ID `<C,0>`.
4. C creates and schedules `ReadTransaction(<C,0>, index=3, destination=R2)`.
5. `start()` sends `ReadMsg(<C,0>, sender=C, index=3)`.
6. The FSM enters `WAITING_RESULT` and schedules `ReadTimeoutMsg(<C,0>)` locally.
7. R2’s receive builder matches `ReadMsg` and calls `onReadMsg`.

8 R2 reads its local `positions[3]`, suppose 44.

9 R2 replies with `ReadResultMsg(<C,0>, value=44, replicaId=2)`.

10 C routes it to the current transaction by ID.

11 The transaction cancels the timeout, enters `DONE`, invokes `callbackOnReadResult`, and completes.

12 C starts the next queued request, if any.

The replica does not create a `ReadTransaction`; only the client owns this FSM. The replica handles the request as one immediate operation.

5. Timeout trace

Now suppose R2 is already crash-simulated.

1 C sends the same `ReadMsg` and schedules the timeout.

2 R2 receives or has the message delivered, but its crashed behavior sends no reply.

3 Akka scheduler later enqueues `ReadTimeoutMsg(<C,0>)` to C.

4 C routes the timeout to the waiting transaction.

5 The transaction enters `TIMEOUT` and invokes `callbackOnReadTimeout(client=C, replica=R2, index=3)`.

6 The next client operation can start.

The timeout is local and intentionally does not use the network channel.

6. Sequential consistency meaning

Reads return local replica state. If one client always sends operations to R2 and the client queue preserves order, that client sees R2's values in R2's application order.

This is not linearizability. A read on R3 can return an older value while a valid update is still in transit. The specification accepts this local viewpoint while requiring eventual ordered application of writes.

7. Index validation issue

`Replica` provides `getPosition(index)`, which checks that the index is between zero and the fixed array length minus one. The inspected read branch's `onReadMsg` directly accesses `positions[msg.index]` instead.

For a valid course request, behavior is the same. For an invalid index, direct access can throw `ArrayIndexOutOfBoundsException` rather than following the explicit validation policy. This is a real implementation detail to preserve in documentation, not a feature of the intended protocol.

The specification does not clearly define invalid-client-index semantics, so a final implementation should state whether invalid input is rejected, times out, or returns a failed result.

8. Epoch metadata

The read branch creates `ReadMsg` with a null epoch pair and copies that metadata into the result. This is reasonable if reads are deliberately outside update ordering, but it remains a contract decision that should be explicit.

A read does not need an epoch pair to return a local value. It may still be useful to include the replica's observed epoch in a richer API, but that is beyond the supplied callback contract.

9. Stale result behavior

Suppose the timeout fires, the transaction completes, and then a delayed result arrives. The client now has no matching current ID—or has advanced to a newer ID—so `Client.onMessage` discards the result.

That prevents two callbacks for one request. It also means a timeout cannot later be “corrected” into success.

10. Tests and their limits

`TestReadTransaction.testReadTransaction` sets a replica’s position to 35, sends a read, and checks success, index, and value.

`testReadTransactionTimeout` crashes the target and checks the timeout’s index, replica, and client.

Useful missing cases include:

- verifying the returned `fromReplica` field in the success test;
- invalid negative and too-large indexes;
- a late result after timeout;
- several queued reads preserving order;
- reading while updates are being applied; and
- running the feature in the final integrated branch.

11. Code map

- `AbstractClient.ReadRequest`: public entry object.
- `Client.sendRead`: constructs and schedules the FSM.
- `ReadTransaction.start`: sends request and schedules timeout.
- `Replica.onReadMsg`: performs local lookup and replies.
- `ReadTransaction.computeState`: handles result or timeout.
- `AbstractClient.callbackOnReadResult/Timeout`: observable output.
- `TestReadTransaction`: focused branch tests.
- `charts/Transactions/transaction_Read.mmd`: success and crash sequences.

12. Exam rehearsal

Why does the replica not ask the coordinator? The project specifies local reads and sequential consistency per replica, so coordination would add cost beyond the required contract.

Why does the timeout carry the transaction ID? It must return through the same client dispatcher and affect only the request that scheduled it.

The invariant to remember is: **a read returns exactly one local snapshot result or one timeout callback for the currently active client transaction.**

13. Read FSM as a traceable state machine

```
INIT --start--> WAITING_RESULT --ReadResultMsg--> DONE
                        |
                        +--------ReadTimeoutMsg--------> TIMEOUT

DONE/TIMEOUT: cancel timer, callback once, remove active transaction
```

The read protocol is intentionally smaller than the write protocol. It sends one request to a chosen replica and waits. “Local read” means the selected replica reads its own materialized array; it does not mean the client bypasses the transaction framework or that every replica already has the same value.

14. Code microscope: request, validation, completion

Read the implementation in this order:

1. **Construction:** record client, target replica, index, and transaction ID.
2. **Start:** schedule a timeout and send `ReadMsg` to the target.
3. **Replica handler:** validate the message belongs to this read and obtain the value at the requested index.
4. **Result handler:** stop accepting duplicate results, cancel the timer, and call `callbackOnReadResult`.
5. **Timeout handler:** use the same cleanup path but call `callbackOnReadTimeout`.

In the read feature branch, the replica-side `onReadMsg` directly indexes the positions array in one path. That is a useful code-review lesson: a protocol can have a correct happy-path FSM while still lacking input validation. A negative or too-large index should produce a controlled failure/timeout rather than an accidental `ArrayIndexOutOfBoundsException`.

```
// Safer conceptual boundary
if (index < 0 || index >= positions.length) {
    replyInvalidIndexOrTimeout(msg.transactionId());
    return;
}
reply(new ReadResultMsg(msg.transactionId(), positions[index]));
```

15. Sequential consistency and stale results

If one client queues `write(x=7)` followed by `read(x)`, the client queue prevents the read from starting before the write transaction completes. If a different client reads a lagging follower, the result can still be older; that is a system-level replication question, not a read-FSM bug. During recovery, the read target should also be checked against the protocol's "safe to serve" state if the specification forbids reads from a stale replica.

Practice

Add tests for invalid index, duplicate `ReadResultMsg`, result-after-timeout, and a read sent to a crashed target. For each, state whether the expected outcome is a callback, a timeout, or silent stale-message rejection.